

MODULE 1 ANSWERS

1. Epipolar geometry and RANSAC

Epipolar geometry is the intrinsic projective geometry between two views of the same scene. It relies solely on the internal parameters of the cameras and their relative position and orientation, remaining independent of the actual scene structure. This geometry provides the fundamental constraints required for stereo vision and 3D reconstruction.

Key Concepts and Explanations:

Definition of the Epipolar Plane: The epipolar plane is a geometric plane defined by three specific points: the optical center of the left camera, the optical center of the right camera, and the 3D point in the scene being viewed. This plane cuts through the image planes of both cameras.

The Epipoles: The epipole is the specific point on an image plane where the baseline (the line connecting the two camera centers) intersects the image. Effectively, the epipole in the left image is the image of the right camera's optical center. All epipolar lines in an image converge at this single point.

Epipolar Lines and Constraint: The intersection of the epipolar plane with the image plane forms a straight line called the epipolar line. The fundamental epipolar constraint states that for a selected point in the first image, its corresponding matching point in the second image must lie on the corresponding epipolar line. This is a critical geometric truth used in stereo matching.

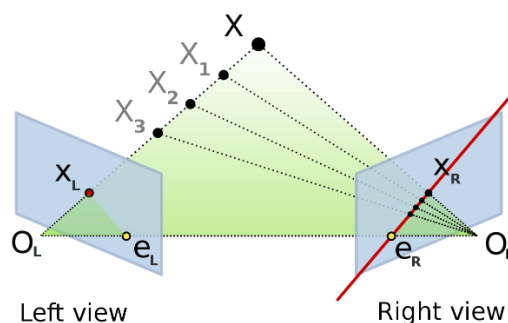
Search Space Reduction: In a naive stereo matching approach, one would search the entire second image to find a match for a pixel in the first image (a 2D search). Using epipolar geometry, the search is restricted to just the epipolar line (a 1D search). This drastically reduces computational cost and the probability of false matches.

The Fundamental Matrix: The algebraic representation of this geometry is the Fundamental Matrix, a 3 by 3 matrix. It encapsulates the intrinsic calibration of both cameras and the rigid motion between them. If a point in one image is multiplied by this matrix and the transpose of the point in the other image, the result is zero.

Role of RANSAC in Feature Matching: In real-world robotic vision, feature matching (connecting points between two images) is error-prone. Repetitive textures, such as foliage or windows, often cause algorithms to link unrelated points, creating outliers. Standard mathematical fitting methods fail in the presence of these outliers.

Random Sample Consensus (RANSAC) Process: RANSAC is an iterative algorithm designed to robustly estimate the Fundamental Matrix despite these errors. It starts by randomly selecting the minimum number of point matches required to solve the geometry (typically seven or eight points). It assumes these few points are correct and computes a temporary model.

Verification and Refinement: The algorithm then tests every other match in the dataset against this temporary model. Matches that fit the model within a strict error threshold are counted as inliers. This process is repeated many times with different random selections. The model that explains the largest number of matches (has the most inliers) is chosen as the correct geometry. Finally, the matrix is re-calculated using all identified inliers to maximize accuracy, effectively ignoring the mismatched outliers.



2. DL-based segmentation in unstructured environments

Deep learning-based segmentation models have revolutionized how robots perceive complex, unstructured environments like forests, planetary surfaces, or disaster zones. Unlike structured roads with clear lane markings, these environments require a dense, pixel-level understanding of terrain and obstacles.

Key Concepts and Explanations:

Semantic Segmentation Definition: Semantic segmentation involves assigning a specific class label, such as rock, sand, grass, or water, to every single pixel in an input image. The output is not a bounding box but a dense classification map that delineates the boundaries of different regions.

Fully Convolutional Networks (FCN): Traditional Convolutional Neural Networks (CNNs) used for classification typically end with fully connected layers that discard spatial information. FCNs replace these layers with convolutional ones, allowing the network to accept images of any size and output a spatial map of class probabilities, preserving the 2D structure of the scene.

Encoder-Decoder Architecture (U-Net): The most common architecture for robotic segmentation is the U-Net. It consists of two parts. The Encoder (or contracting path) progressively downsamples the image to extract high-level semantic context, answering the question "what is in the image."

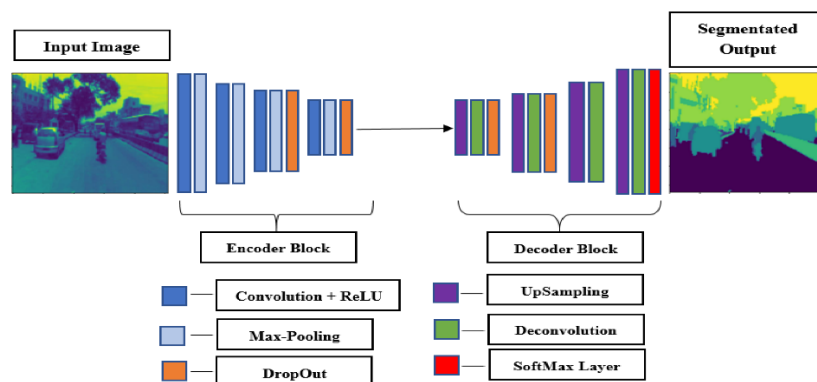
The Decoder and Localization: The Decoder (or expanding path) takes the high-level features and progressively upsamples them back to the original image resolution. This recovers the spatial information, answering the question "where is it." This symmetry allows for precise boundary definition.

Importance of Skip Connections: In deep networks, spatial details like the precise edges of a small rock can be lost during pooling operations in the encoder. U-Net introduces skip connections that copy high-resolution feature maps from the encoder layers directly to the corresponding decoder layers. This allows the network to combine semantic context with fine-grained spatial details.

Instance Segmentation for Manipulation: While semantic segmentation groups all objects of the same class (e.g., all rocks) into one blob, Instance Segmentation (using models like Mask R-CNN) detects and delineates each distinct object separately. This is crucial if a robot needs to pick up a specific stone or avoid a specific branch.

Application to Traversability Analysis: In unstructured environments, geometry alone is often insufficient. A patch of tall grass and a solid concrete block might look geometrically similar (both are raised). Deep learning segmentation uses texture and color to distinguish traversable vegetation from non-traversable obstacles.

Adaptability to Environmental Hazards: These models allow robots to identify arbitrary hazards that lack standard shapes. For example, in a Mars rover scenario, segmentation helps identify loose drift sand which could trap the rover, distinguishing it from reliable bedrock, a task that traditional edge detection would fail at.



3. Image morphing vs image transformation

While both image morphing and image transformations involve manipulating images, they differ fundamentally in their goals, inputs, and underlying mechanisms. Transformations are geometric operations on a single image, while morphing is a temporal transition between two images.

Key Concepts and Explanations:

Definition of Image Transformation: An image transformation is a geometric mapping of pixel coordinates from a source domain to a destination domain. It changes the spatial arrangement of the image content without necessarily adding new visual information.

Input and Output for Transformations: The input is typically a single source image. The output is a modified version of that same image. Common examples include rotation, scaling, translation, and perspective warping (homography).

Mechanism of Transformation: The process involves a mathematical function or matrix that maps every coordinate (x, y) in the input image to a new coordinate $(x\text{-prime}, y\text{-prime})$ in the output. The pixel intensity or color values are essentially moved to new locations, though interpolation is used to handle non-integer coordinates.

Purpose of Transformation: The primary goal is geometric alignment or view synthesis. For example, rectifying a scanned document to make it flat, or rotating a map to align with the robot's heading. The semantic content of the image remains identical; only its geometry changes.

Definition of Image Morphing: Image morphing is a visual effects technique used to generate a smooth, seamless transition sequence between two different images. It creates the illusion that one object is fluidly transforming into another.

Input and Output for Morphing: The input requires two distinct images: a Source Image and a Destination Image. The output is a sequence of frames (an animation) that bridges the gap between them.

Mechanism of Morphing (Dual Process): Morphing combines two simultaneous operations. First, Geometric Warping distorts the shape of the source object to match the shape of the destination object. Second, Cross-Dissolving (Photometric Interpolation) fades the color values of the source out while fading the destination values in.

Feature Correspondence and Example: To achieve a convincing morph, control points must be defined on both images (e.g., matching the eyes of a human to the eyes of a tiger). An example is an animation where a person's face transforms into an animal; the facial structure physically expands or shrinks (warp) while the skin texture changes color and pattern (dissolve) at the same time.

4. Single-view geometry and camera model development

Single-view geometry deals with recovering 3D structural information or internal camera parameters from a single 2D image. It relies on exploiting geometric regularities naturally present in man-made environments, such as parallel lines and orthogonal planes, rather than using stereo parallax.

Key Concepts and Explanations:

Vanishing Points: In perspective projection, parallel lines in the 3D world do not remain parallel in the 2D image; instead, they converge to a single point known as the vanishing point. This is a fundamental cue in single-view geometry.

The Horizon or Vanishing Line: The line connecting two vanishing points corresponds to the horizon line for the plane defined by those directions. For example, the vanishing points of two horizontal edges on a floor will define the horizon of that floor.

Orthogonality Constraints: If a scene contains three mutually orthogonal directions (like the X, Y, and Z axes formed by the corner of a room), their three corresponding vanishing points impose strong mathematical constraints. The vectors from the camera center to these vanishing points on the image plane must be perpendicular in 3D space.

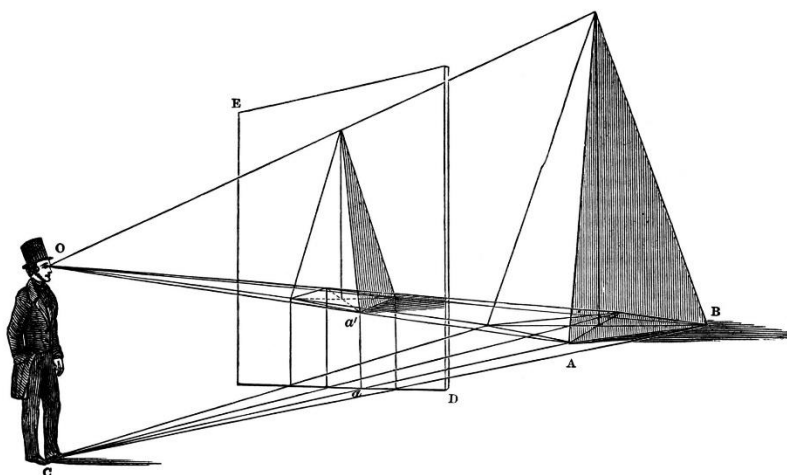
Camera Calibration from Scene: This geometry is vital for camera model development because it allows for self-calibration. By identifying three orthogonal vanishing points, one can form equations to solve for the camera's intrinsic parameters, specifically the focal length and principal point, without needing a laboratory calibration checkerboard.

The Image of the Absolute Conic (IAC): Theoretically, the vanishing points of orthogonal directions lie on a virtual geometric entity called the Image of the Absolute Conic. Identifying this conic allows for the direct computation of the calibration matrix K , which is the core of the pinhole camera model.

Metric Reconstruction: Once the camera parameters are recovered via single-view geometry, the perception of the scene can be upgraded from Projective to Metric. This means the system can now measure true angles between rays and ratios of lengths, which is impossible in an uncalibrated projective view.

Application in Metrology: Single-view geometry enables metrology, or taking measurements from images. For instance, if the height of a reference object (like a door frame) is known, the height of other objects (like a person) can be calculated using the cross-ratio of distances relative to the vanishing line.

Scene Layout Understanding: For indoor robotics, single-view geometry allows the robot to infer the 3D layout of a room (walls, floor, ceiling) from a single snapshot by assuming the "Manhattan World" constraint, where mostly everything is built on three orthogonal axes.



5. Object detection pipeline with segmentation and RANSAC

This pipeline represents a robust, classical computer vision approach for identifying specific objects in a robotic scene. It moves from coarse region identification to precise pose estimation, using outlier rejection to ensure reliability.

Key Concepts and Explanations:

Step 1: Segmentation and Region Proposal: The first step aims to reduce the search space. Instead of searching the entire high-resolution image, the system isolates Regions of Interest (ROIs). This is often done using computationally cheap segmentation methods like background subtraction (if the camera is static) or color-based thresholding (e.g., finding all yellow blobs for a specific tool).

Step 2: Feature Extraction: Once an ROI is identified, the pixels must be converted into a mathematical format that is recognizable. Algorithms like SIFT (Scale-Invariant Feature Transform) or ORB are used to detect keypoints (distinct corners or blobs) and compute descriptors. These descriptors are vectors that describe the local texture and are invariant to changes in scale, rotation, and lighting.

Step 3: Feature Matching: The robot compares the descriptors from the live view against a stored database of the object model. A Nearest Neighbor search finds the closest match for each point.

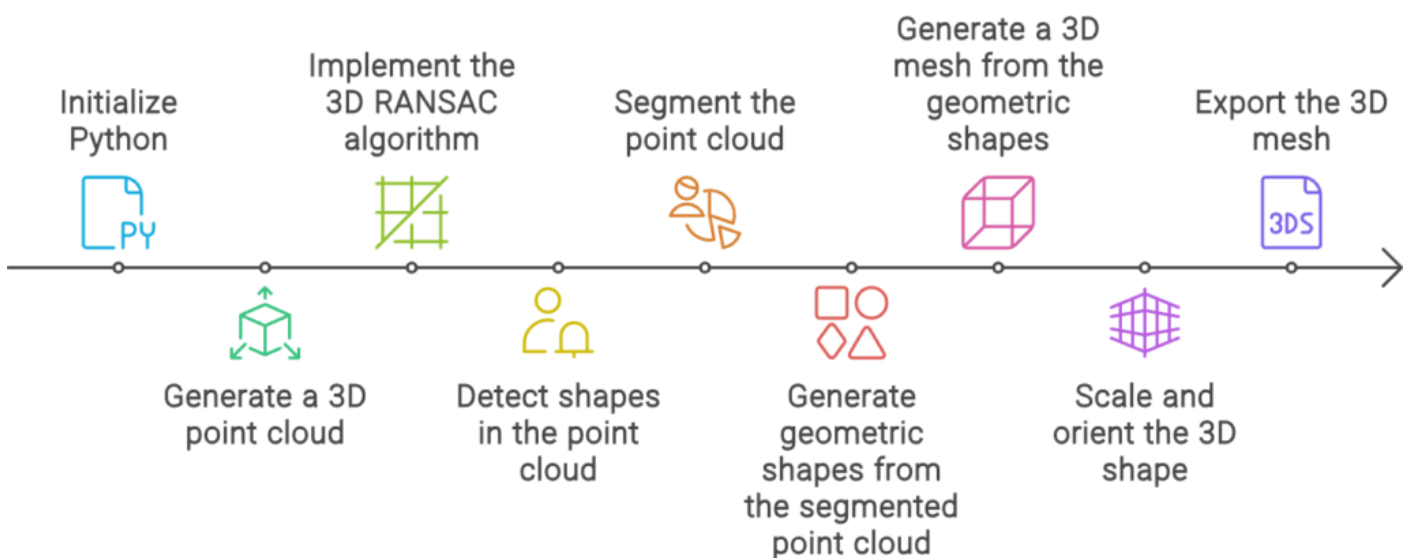
Step 4: Filtering Ambiguity (Ratio Test): Not all matches are good. Lowe's Ratio Test is applied, which accepts a match only if the distance to the best neighbor is significantly smaller than the distance to the second-best neighbor. This eliminates distinctive features that are actually ambiguous or repetitive.

Step 5: RANSAC for Geometric Verification: Even after the ratio test, some matches will be incorrect (outliers). RANSAC (Random Sample Consensus) is used to filter these. It randomly selects a small subset of matches (e.g., four) and computes a hypothetical geometric transformation (like a Homography) that aligns the object model to the scene.

Step 6: Outlier Rejection: The algorithm tests the remaining matches against this hypothesis. Matches that align physically with the transformation are kept as inliers; those that physically deviate are discarded as outliers.

Step 7: Pose Estimation: The largest cluster of consistent inliers is used to compute the final position and orientation (pose) of the object. This 6-Degree-of-Freedom pose tells the robot exactly where the object is relative to the camera.

Step 8: Refinement: Finally, a non-linear least squares optimization is often performed using all the validated inliers. This refines the pose estimate to sub-pixel accuracy, ensuring the robot can precisely grasp or manipulate the object.



6. Calibration and single-view geometry

Camera calibration is the process of determining the internal characteristics of a camera, and it plays a decisive role in the accuracy and capability of single-view geometry algorithms.

Key Concepts and Explanations:

Definition of Calibration Matrix K: Calibration estimates the Intrinsic Matrix, denoted as K . This matrix contains the focal length (in horizontal and vertical pixels), the principal point (the optical center on the image), and potentially a skew parameter. It defines the mapping from the 3D camera coordinate system to the 2D pixel coordinate system.

Lens Distortion Correction: Real robotic cameras, especially wide-angle ones, suffer from radial distortion, where straight lines appear curved (barrel effect). Single-view geometry algorithms rely heavily on the assumption that straight lines in the world project to straight lines in the image to find vanishing points. Calibration provides the coefficients to undistort the image, making these algorithms viable.

Projective vs. Euclidean Stratification: Without calibration, a single view only exists in Projective space. In this space, one can determine incidence (if a point lies on a line) but cannot measure angles or distances. Calibration upgrades the space to Euclidean, enabling metric measurements.

Measuring Angles Between Rays: With a calibrated camera, a 2D pixel coordinate can be back-projected into a normalized 3D direction vector (ray). The angle between any two such rays can be computed using the dot product formula. This is essential for checking if two observed walls are perpendicular.

Determining Plane Orientation: If the vanishing line (horizon) of a plane is detected, calibration allows the robot to compute the 3D normal vector of that plane. This tells the robot the exact orientation of the surface (e.g., is the ramp steep or flat?) relative to the camera.

Metric Ratios and Height Estimation: While single-view geometry can provide cross-ratios without calibration, extracting real-world metric ratios (like height in meters) often requires the focal length to be known to relate pixel distances to real-world viewing angles.

Robustness to Center Shift: The principal point is not always at the exact center of the image due to sensor mounting errors. Calibration identifies the true optical center. Single-view algorithms that assume the center is at $(width/2, height/2)$ will produce biased vanishing points if the camera is uncalibrated.

Integration with IMU: In robotics, the calibration matrix allows for the fusion of visual data with Inertial Measurement Units (IMUs). The IMU provides gravity vectors in metric space; calibration allows the visual vertical vanishing point to be aligned with the gravity vector, correcting the robot's tilt estimate.

7. Filtering and restoration for noisy datasets

Robotic vision systems often operate in less-than-ideal conditions, resulting in noisy image datasets. A structured, stepwise pipeline is required to clean the data while preserving the critical information needed for perception tasks.

Key Concepts and Explanations:

Step 1: Noise Identification and Analysis: Before applying any filter, the noise model must be identified. Statistical analysis of homogeneous image patches can reveal if the noise is Gaussian (thermal/sensor noise, appearing as grain) or Impulse (Salt-and-Pepper, appearing as random black and white dots).

Step 2: Non-Linear Filtering (Median Filter): The first stage of cleaning is usually outlier removal. A Median Filter is chosen because it is non-linear and excellent at removing salt-and-pepper noise. Unlike a mean filter which smears the outlier into surrounding pixels, a median filter replaces the pixel with the median value of its neighbors, completely eliminating the spike while preserving sharp edges.

Step 3: Edge-Preserving Smoothing (Bilateral Filter): For the remaining Gaussian noise, a smoothing filter is needed. However, standard Gaussian blurring softens edges, which ruins object detection. A Bilateral Filter is used instead. It weighs pixels based on both spatial closeness and intensity similarity. This means it smooths pixels on flat surfaces (walls) but stops at edges where intensity changes abruptly, preserving the structural details.

Step 4: Deconvolution for Motion Blur: If the robot is moving, the images likely suffer from motion blur. Restoration involves Deconvolution. If the blur pattern (Point Spread Function) is estimated, Wiener Filtering is applied. This mathematically inverts the blur operation.

Step 5: Noise Control in Deconvolution: Pure inverse filtering amplifies high-frequency noise. Wiener filtering is superior because it incorporates the Signal-to-Noise Ratio (SNR) into the equation, performing deblurring only where the signal is strong enough, thus preventing the restoration process from creating a noisy mess.

Step 6: Contrast Enhancement (CLAHE): After cleaning, the image might still be low-contrast due to poor lighting. Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied. It enhances contrast locally in small tiles of the image.

Step 7: Preventing Noise Amplification in Contrast: Standard histogram equalization can make background noise very prominent in uniform areas. The "Contrast Limited" part of CLAHE clips the histogram at a specific threshold, ensuring that noise is not amplified during the enhancement process.

Step 8: Final Output Verification: The final stage is often a check for artifacts. The processed image is passed to the next stage of the vision pipeline (e.g., an edge detector). If the edge detector finds too many spurious edges, the parameters of the smoothing steps (like the sigma in the Bilateral filter) are adjusted in a feedback loop.

MODULE 2 ANSWERS

8. Visual odometry: semi-direct vs direct sparse

Visual Odometry is the process of estimating the egomotion of an agent using only the input of a single or multiple cameras. Two prominent approaches to this are Semi-Direct Visual Odometry and Direct Sparse Odometry, which differ fundamentally in how they process image data.

Key Concepts and Explanations:

Definition of Semi-Direct Visual Odometry (SVO): SVO is a hybrid algorithm that combines the computational speed of direct methods with the consistency of feature-based methods. It splits the tracking and mapping tasks into different methodologies.

Tracking in SVO (Direct Method): For frame-to-frame motion estimation, SVO uses a direct approach. It minimizes the photometric error (pixel intensity difference) for patches of pixels surrounding detected corners. It does not compute expensive feature descriptors like SIFT or ORB. Instead, it assumes that the intensity of a pixel remains constant between frames and aligns the camera to minimize the brightness difference.

Mapping in SVO (Feature-based Method): Once the camera pose is roughly estimated, SVO switches to a feature-based approach for mapping. It uses probabilistic depth filters to estimate the 3D coordinates of the tracked corners. This split allows SVO to be extremely fast, often running at high frame rates suitable for onboard drone processing.

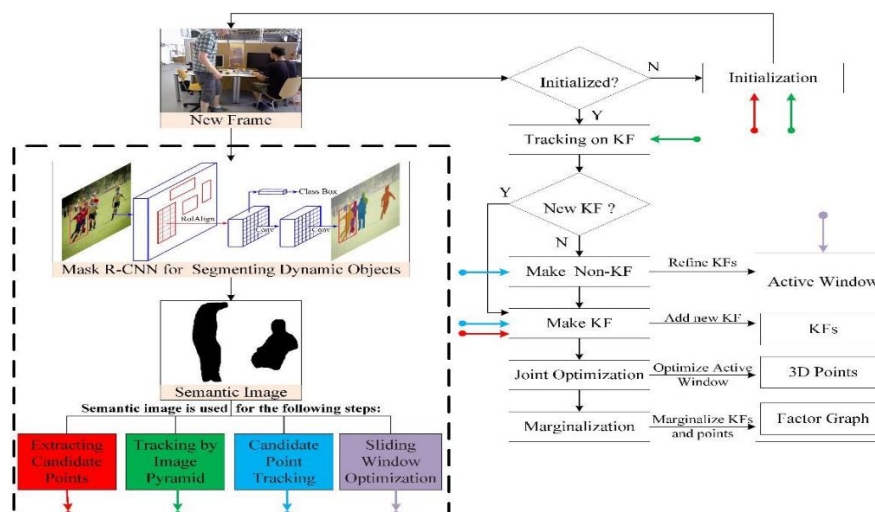
Definition of Direct Sparse Odometry (DSO): DSO is a fully direct method that abandons keypoints entirely. It operates on pixel intensities throughout the entire pipeline, both for tracking and mapping.

Sparse Sampling Strategy: Unlike dense methods that use every pixel in the image, DSO selects a sparse set of pixels to process. However, unlike feature-based methods that select corners, DSO selects any pixel with a high intensity gradient. This includes edges, shadows, and smooth gradients on curved surfaces.

Joint Optimization: DSO performs a joint optimization of all model parameters. It minimizes a photometric error function over a sliding window of recent keyframes. It simultaneously refines the camera pose, the inverse depth of all points, and even internal camera parameters like affine brightness parameters (exposure time and gain).

Robustness Comparison: SVO works best in environments with distinct corners (cluttered rooms). DSO is more robust in low-texture environments where corners are scarce but gradients exist, such as asphalt roads or smooth concrete walls.

Example Applications: SVO is widely used in micro-aerial vehicles (drones) where low latency is critical for flight stability. DSO is often applied in autonomous driving or handheld augmented reality where lighting conditions vary and objects may lack sharp corners.



9. Bundle adjustment and sparsity

Bundle Adjustment is the final refinement step in most visual mapping systems. It is a large-scale optimization problem that ensures the consistency of the estimated map and trajectory.

Key Concepts and Explanations:

Definition of Bundle Adjustment: Bundle Adjustment is the problem of refining a visual reconstruction to produce jointly optimal 3D structure and viewing parameter estimates. It adjusts the "bundle" of light rays leaving each 3D feature and converging on each camera center.

The Objective Function: The goal is to minimize the total Reprojection Error. This is the sum of squared distances between the observed 2D point locations in the images and the projected positions of the estimated 3D world points.

Optimization Method: Because the relationship between 3D points and 2D pixels involves perspective division (non-linear), Bundle Adjustment is solved using non-linear least squares algorithms, most commonly the Levenberg-Marquardt algorithm. This algorithm iteratively improves the estimate by interpolating between the Gauss-Newton method and Gradient Descent.

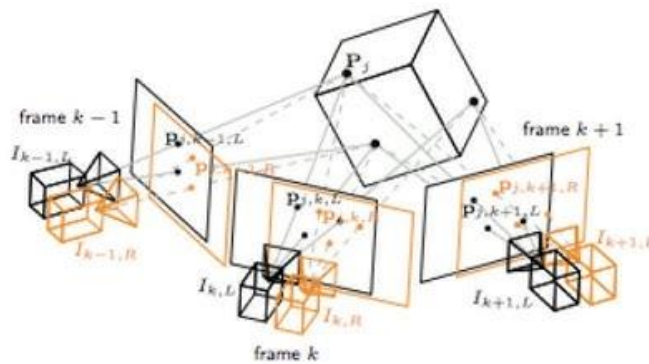
Computational Challenge: The primary difficulty is the size of the problem. If a map has thousands of cameras and millions of points, the matrix required to solve the update steps (the Hessian matrix) becomes enormous. Solving a dense linear system of this size would be computationally impossible.

The Concept of Sparsity: The key to solving this efficiently is recognizing that the Hessian matrix is sparse, meaning it is mostly filled with zeros. This sparsity arises from the physical structure of the problem.

Why the Matrix is Sparse: A specific camera only sees a small fraction of the total 3D points in the world. Conversely, a specific 3D point is only seen by a few cameras. There is no direct interaction between two 3D points; Point A and Point B are independent of each other given the camera poses. This creates a block-diagonal structure in the matrix.

The Schur Complement: To exploit this sparsity, a mathematical trick called the Schur Complement or Marginalization is used. This allows the system to algebraically eliminate the 3D point parameters from the equation temporarily.

Resulting Efficiency: By eliminating the points, the solver is left with a much smaller system known as the Reduced Camera System, which only involves the camera parameters. Once the camera poses are solved, the 3D point positions can be computed efficiently via back-substitution. This reduces the complexity from cubic in terms of total parameters to cubic in terms of the number of cameras, making large-scale mapping feasible.



$$\min_{\mathbf{a}_j, \mathbf{b}_i} \sum_{i=1}^n \sum_{j=1}^m v_{ij} d(\mathbf{Q}(\mathbf{a}_j, \mathbf{b}_i), \mathbf{x}_{ij})^2$$

10. Re-localization and large-scale map optimization

In Simultaneous Localization and Mapping (SLAM), building a map is only half the battle. The system must also maintain the map's consistency over long periods and recover when tracking is lost.

Key Concepts and Explanations:

The Re-localization Challenge: Re-localization is the ability of a robot to identify its position in a previously mapped environment without relying on its current motion estimate. This is often called the Kidnapped Robot Problem.

Perceptual Aliasing: A major difficulty in re-localization is perceptual aliasing, where different locations look identical. For example, in a large office building, many corridors have the same white walls and grey carpet. A visual search might incorrectly match the current view to the wrong corridor, causing the robot to teleport in the map.

Place Recognition Techniques: To solve this, SLAM systems use Bag of Words (BoW) approaches. The image is converted into a histogram of visual words (quantized feature descriptors). This allows for extremely fast database queries to find the most visually similar past keyframe.

Geometric Verification: Once a candidate past frame is found, geometric verification is essential. The system attempts to match 2D features in the current view with 3D points from the candidate map section using a Perspective-n-Point (PnP) solver. If a valid pose is found, re-localization is successful.

Drift in Large-Scale Mapping: As a robot explores a large area, small errors in frame-to-frame tracking accumulate. Over time, this drift causes the map to warp. A long straight corridor might appear curved, or a building might not close when the robot walks around it.

Loop Closure: The solution to drift is Loop Closure. This occurs when the robot recognizes a location it has visited before. This detection creates a new constraint that ties the current pose to an old pose, effectively asserting that these two distinct points in time correspond to the same place in space.

Pose Graph Optimization: In large-scale systems, running full Bundle Adjustment on the entire map to correct drift is too slow. Instead, Pose Graph Optimization is used. The 3D points are frozen, and the system optimizes a graph where nodes are camera poses and edges are relative motion constraints.

Optimization Effect: When a loop is closed, the optimization algorithm distributes the accumulated error backward along the robot's trajectory, like bending a flexible wire to make the ends meet. This snaps the map into a consistent global shape without needing to re-process every single pixel.

11. SLAM: initialization, tracking and mapping

A Visual SLAM system operates in three distinct, interconnected stages. Each stage has specific goals and mathematical tools.

Key Concepts and Explanations:

Step 1: Initialization: The goal of initialization is to establish the global coordinate frame and the initial map scale. For monocular systems, this is challenging because a single image has no depth information.

The Initialization Process: The robot must move to create parallax. The system tracks features across the first two frames and computes the Essential Matrix or Homography. From this, it decomposes the rotation and translation matrices and triangulates the initial set of 3D points. This sets the base unit for the map (e.g., the distance moved is defined as 1 unit).

Step 2: Tracking (Front-End): The tracking thread runs in real-time for every incoming frame. Its responsibility is to estimate the current camera pose relative to the existing map.

Tracking Mechanism: The system predicts the camera's position using a constant motion model. It then projects the local 3D map points into the current image frame. It searches for matching feature points near these projections. Finally, it minimizes the reprojection error (Motion-only Bundle Adjustment) to refine the camera pose.

Step 3: Mapping (Back-End): The mapping thread runs at a lower frequency and focuses on expanding and refining the world model.

Keyframe Selection: Not every video frame is added to the map. To prevent data redundancy and memory bloat, the system only selects Keyframes, which are frames where the robot has moved significantly or seen new areas.

Triangulation of New Points: The mapping thread looks for feature points that are visible in the new keyframe and its neighbors but are not yet in the map. It triangulates these to create new 3D landmarks.

Local Bundle Adjustment: To ensure accuracy, the mapping thread performs Local Bundle Adjustment. It optimizes the poses of the last N keyframes and the positions of the local 3D points, smoothing out jitter and correcting small drift errors before they accumulate.

Example Scenario: Consider a vacuum robot. It starts (Initialization) by moving slightly to understand the floor depth. As it moves (Tracking), it follows the wall, matching floor patterns to its internal map. When it enters a new room (Mapping), it creates a new keyframe and adds the furniture locations to its memory.

12. Sensor characteristics for SLAM

The choice of sensor dictates the algorithms used for SLAM and the environments in which the robot can operate. The three primary visual sensors are Monocular, Stereo, and RGB-D.

Key Concepts and Explanations:

Monocular Sensors (Single Camera): These are the simplest and cheapest sensors, consisting of a standard camera.

Scale Ambiguity: The critical limitation is scale ambiguity. A monocular camera cannot distinguish between a large object far away and a small object close up. The trajectory is estimated only up to an unknown scale factor.

Suitability: They are suitable for lightweight platforms like drones or mobile phones where weight and power are constraints. However, they require complex initialization and often need fusion with an Inertial Measurement Unit (IMU) to resolve the scale.

Stereo Sensors (Two Cameras): A stereo rig consists of two calibrated cameras separated by a fixed horizontal baseline.

Absolute Scale: Because the baseline is known, stereo cameras can triangulate absolute depth. There is no scale ambiguity.

Passive Depth Perception: Unlike active sensors, stereo works well outdoors in direct sunlight.

Suitability: They are the standard for outdoor robotics, such as autonomous cars and planetary rovers. Their main limitation is that depth accuracy degrades with distance; they essentially become monocular cameras for objects very far away.

RGB-D Sensors (Depth Cameras): These combine a standard RGB camera with an active depth sensor, typically using structured light (projecting an infrared pattern) or Time-of-Flight (measuring light pulse return time).

Dense Depth: They provide a dense, accurate depth value for every pixel in the image instantly, without needing motion or texture.

Texture Independence: They work perfectly on featureless surfaces like white walls, where monocular and stereo methods fail.

Suitability: They are ideal for indoor service robots. However, they are generally unsuitable for outdoor use because sunlight overwhelms the infrared projector, and they have a limited maximum range (typically less than 10 meters).

13. Direct vs indirect SLAM; LSD-SLAM and ORB-SLAM

SLAM formulations are broadly categorized into Indirect (Feature-based) and Direct methods based on how they process image information.

Key Concepts and Explanations:

Indirect (Feature-Based) Formulation: These methods abstract the image into sparse geometric features like corners or blobs. They extract these features first and then discard the rest of the image. The optimization objective is to minimize Geometric Error, which is the distance in pixels between where a feature was detected and where the map point projects.

Strengths of Indirect Methods: They are robust to lighting changes and wide baselines because feature descriptors (like SIFT or ORB) are designed to be invariant to these changes. They are generally more efficient because they process less data.

Representative System (ORB-SLAM): ORB-SLAM is the premier feature-based system. It uses ORB features for tracking, mapping, and loop closure. Its primary strength is robustness; it can recover from lost tracking and close loops effectively. Its weakness is that the resulting map is a sparse point cloud, which is often hard for humans to interpret or for robots to use for path planning.

Direct Formulation: These methods use the raw pixel intensity values directly in the optimization. They do not extract features. The optimization objective is to minimize Photometric Error, which is the difference in brightness between a pixel in one image and its projection in another.

Strengths of Direct Methods: They can utilize information from all pixels with gradients, including edges and smooth texture variations, not just corners. This allows them to work in environments where feature detectors fail (e.g., corridors with smooth walls). They also provide sub-pixel accuracy.

Representative System (LSD-SLAM): LSD-SLAM (Large-Scale Direct SLAM) is a famous direct monocular system. Its strength is that it produces a Semi-Dense map. Instead of just points, it reconstructs the 3D structure of all edges and boundaries, creating a recognizable 3D model of the environment. Its weakness is sensitivity to lighting changes (as it relies on brightness constancy) and rolling shutter artifacts.

14. Feature-based alignment and pose estimation

Feature-based alignment techniques are used to estimate the position and orientation (pose) of a camera or object by matching geometric features. These techniques are categorized based on the dimensionality of the input data.

Key Concepts and Explanations:

Category 1: 2D-2D Alignment (Image to Image): This case arises when matching points between two 2D images without prior depth information.

Technique: The goal is to estimate the Epipolar Geometry (Fundamental Matrix) or a Homography (if the scene is planar).

Example: This is used during the Initialization phase of Visual Odometry. The robot compares the first two frames of video to determine the direction and magnitude of its initial movement. The Five-Point Algorithm or Eight-Point Algorithm is commonly used here inside a RANSAC loop.

Category 2: 3D-2D Alignment (World to Image): This case arises when the robot has a known 3D map and needs to find its pose within that map using a current 2D image.

Technique: This is known as the Perspective-n-Point (PnP) problem. The inputs are 3D coordinates of landmarks and their corresponding 2D pixel coordinates. The output is the 6-Degree-of-Freedom pose (Rotation and Translation) of the camera.

Example: This is the standard Tracking step in SLAM. The robot projects its known 3D map into the camera view and aligns itself to minimize the reprojection error. Algorithms like EPnP or P3P are used.

Category 3: 3D-3D Alignment (World to World): This case arises when aligning two 3D point clouds, such as those from a stereo camera or an RGB-D sensor.

Technique: The standard algorithm is Iterative Closest Point (ICP). It iteratively finds the nearest neighbors between two point sets and computes the rigid body transformation that aligns them.

Example: This is used for Loop Closure in RGB-D SLAM. When a robot returns to a previous area, it has two independent 3D maps of the same room. ICP fuses these two maps into a single consistent model.

15. Geometric error formulation in visual SLAM

The choice of error formulation defines the objective function of the SLAM optimization. It determines what the system considers "accurate."

Key Concepts and Explanations:

Indirect (Geometric) Error Formulation: This is the standard for feature-based approaches like ORB-SLAM.

Definition: It measures the Euclidean distance in the image plane. Specifically, it is the distance between the observed 2D location of a feature point (detected by an algorithm like SIFT) and the projected 2D location of the corresponding 3D map point.

Formula Concept: $\text{Error} = \text{Norm}(\text{Measured_Pixel} - \text{Projected_Pixel})$.

Role: This formulation decouples the geometry from the radiometry (appearance). The optimizer does not care about pixel colors, only about minimizing the distances between points. This makes it robust to auto-exposure changes or lighting shifts, provided the feature descriptor matching was correct.

Direct (Photometric) Error Formulation: This is the standard for direct approaches like LSD-SLAM or DSO.

Definition: It measures the difference in intensity (brightness/color) between pixels. It compares the intensity of a pixel in a reference frame with the intensity of the pixel in the current frame where the 3D point projects.

Formula Concept: $\text{Error} = \text{Norm}(\text{Intensity_at_Reference} - \text{Intensity_at_Projected_Current})$.

Role: This formulation assumes Brightness Constancy (a point looks the same from different angles). It allows the system to exploit sub-pixel information because intensity gradients are continuous. It also allows the system to use edges and shadows which are not distinct "points."

Trade-off: While potentially more accurate, it is highly non-convex (harder to optimize) and sensitive to lighting changes. If a cloud passes over the sun, the pixel intensities change, and the error creates false minima.

MODULE 3 ANSWERS

16. Active learning in robotic vision recognition

Active Learning represents a shift from passive data collection to autonomous data selection. In the context of robotic vision, it addresses the critical bottleneck of data annotation.

Key Concepts and Explanations:

The Data Deluge Problem: Robots equipped with cameras generate terabytes of video footage. In a standard supervised learning pipeline, humans would need to label this data, which is prohibitively expensive and slow. Furthermore, 99% of this data is uninformative (redundant frames of the same empty corridor).

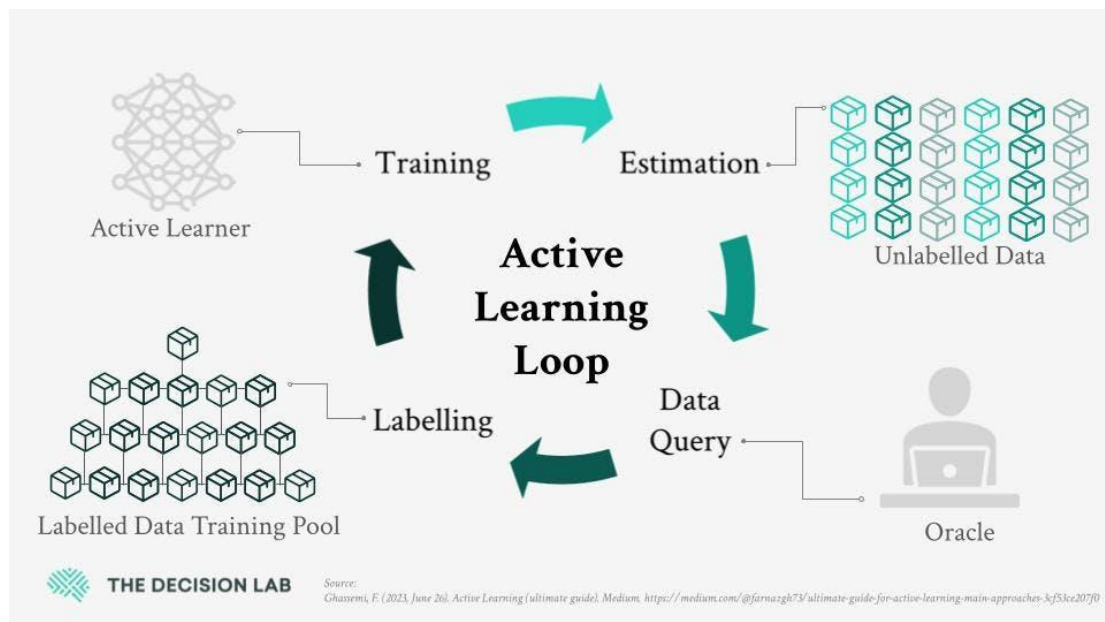
Definition of Active Learning: Active learning is a machine learning strategy where the algorithm itself chooses which data samples are most valuable and requests labels for only those samples.

Uncertainty Sampling Strategy: The most common selection criterion is uncertainty. The robot analyzes its own predictions. If a neural network predicts a "Car" with 99% confidence, the image is ignored. If it predicts "Car" with 51% confidence and "Truck" with 49%, the system flags this frame as "Uncertain."

Querying the Oracle: When a frame is flagged, the robot requests a label from an "oracle" (a human annotator). This loop ensures that human effort is focused exclusively on the edge cases where the model is failing, rather than labeling obvious examples.

Real-Time Application - Novelty Detection: Consider a planetary rover exploring Mars. It is trained to recognize "Sand" and "Basalt." If it encounters a new, shiny mineral, its feature extractor will produce an output far from the known class clusters. Active learning algorithms detect this anomaly and prioritize this specific image for transmission back to Earth, filtering out thousands of images of standard rocks.

Real-Time Application - Domain Adaptation: A delivery robot trained in summer might fail when deployed in winter due to snow. Using active learning, the robot can identify that its confidence drops significantly in snowy regions. It effectively "asks" to learn about snow by selecting those specific frames for retraining, allowing it to adapt to the new season with minimal data.



17. Sequence modelling for vision interpretation

Standard computer vision often treats the world as a bag of unrelated static images. Sequence modelling introduces the dimension of time, which is essential for robotics.

Key Concepts and Explanations:

Temporal Context: Sequence modelling operates on the premise that the current state of the world depends on the previous states. It allows a robot to reason about motion, causality, and persistence.

Recurrent Neural Networks (RNNs) and LSTMs: These are deep learning architectures designed for sequences. They maintain a hidden "memory state." As the robot processes video frame by frame, this memory state is updated, carrying information from the past forward. Long Short-Term Memory (LSTM) networks are a variant designed to remember information over long periods, preventing the "vanishing gradient" problem.

Transformers and Self-Attention: Modern approaches use Transformers. Unlike RNNs which process data sequentially, Transformers use "Self-Attention" mechanisms to look at a whole chunk of video at once. They can relate an event happening at the start of a clip directly to an event at the end, capturing complex long-term dependencies.

Application - Action Recognition: Distinguishing between "Picking up a box" and "Putting down a box" is impossible from a single still frame; the pose is identical. A sequence model analyzes the flow of motion over time to determine the direction of the action.

Application - Trajectory Prediction: For autonomous driving, perceiving a pedestrian is not enough; the car must know where they are going. A sequence model takes the history of the pedestrian's positions ($t-10$ to t) and predicts their future path ($t+1$ to $t+10$), allowing the car to plan a collision-free trajectory.

Application - Object Permanence: If a ball rolls behind a sofa, a standard detector stops seeing it and "forgets" it exists. A sequence model's memory state persists. It "remembers" the ball entered the occlusion zone and can predict it will emerge on the other side, mimicking human-like object permanence.

18. Uncertainty estimation in perception and decision-making

In safety-critical robotics, knowing "what you don't know" is just as important as knowing "what you do know." Uncertainty estimation quantifies the reliability of a robot's perception.

Key Concepts and Explanations:

The Overconfidence Problem: Standard Deep Neural Networks are notoriously overconfident. A network trained only on indoor furniture might classify a tree as a "Lamp" with 99% confidence simply because it is the closest match it knows. This can lead to dangerous robotic behavior.

Aleatoric Uncertainty (Data Uncertainty): This type of uncertainty captures noise inherent in the observations. For example, if a camera is looking through heavy rain, fog, or motion blur, the data itself is corrupted. The model should output high aleatoric uncertainty, effectively saying "I cannot see clearly."

Epistemic Uncertainty (Model Uncertainty): This captures ignorance in the model. It occurs when the input data is clean, but the robot has never seen such an object during training. It represents a lack of knowledge. The model should output high epistemic uncertainty, saying "I have never seen this before."

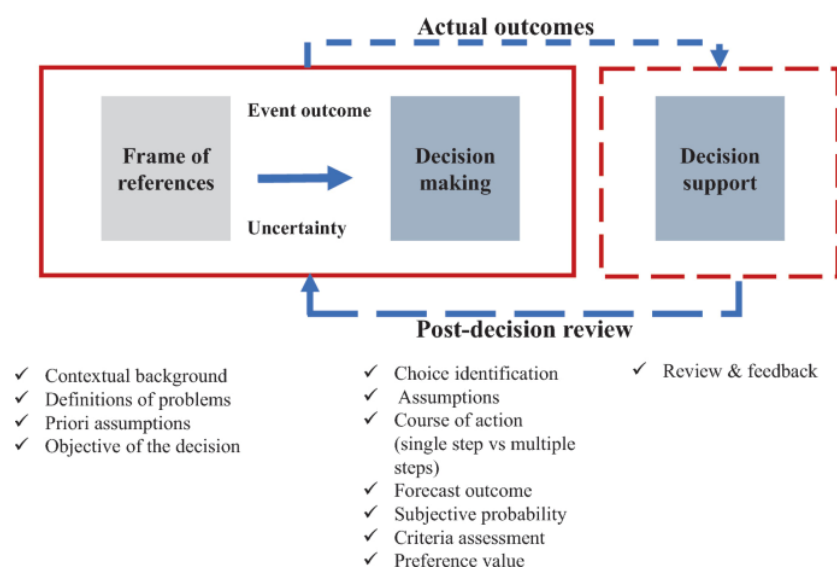
Impact on Sensor Fusion: Uncertainty acts as a dynamic weighting mechanism. If a self-driving car enters a tunnel, the camera's aleatoric uncertainty spikes due to the sudden lighting change. The sensor fusion algorithm detects this and automatically shifts trust to the LiDAR or Radar sensors, which are unaffected by light.

Impact on Decision Making (Control Policy): Uncertainty modulates the robot's aggressiveness.

High Confidence: If a robot arm is 99% sure of an object's position and material, it can execute a fast, ballistic grasp.

High Uncertainty: If the vision system is uncertain about the object's surface friction (e.g., is it wet?), the control policy switches to a cautious approach. The arm moves slowly, makes contact gently, and relies on tactile feedback rather than vision to complete the task.

Bayesian Deep Learning: Techniques like Monte Carlo Dropout are used to estimate this uncertainty. By running the neural network multiple times with random neurons disabled (dropout) during inference, the robot gets a distribution of predictions. The variance of this distribution represents the uncertainty. A high variance means the network is unsure.



https://www.google.com/url?sa=i&url=https%3A%2F%2Flink.springer.com%2Fchapter%2F10.1007%2F978-981-99-5253-3_2&psig=AOvVaw0b1-rCzcCFwFx988sEgUL&ust=1764598959959000&source=images&cd=vfe&opi=89978449&ved=0CBgQjhxqFwoTCIDw0cOJmpEDFQAAAAAdAAAAABAK